

LABORATORY OBSERVATION/RECORD

Course: Full Stack Web Development

Course Code: 23CM4121

Experiment No.: 3

Student Name: V S SRAVAN M

Roll No.: A24126552268

1. TITLE

Working with JavaScript Arrays and Functions

2. AIM

To understand and implement JavaScript data structures (Arrays) and control abstractions (Functions) using ES6+ syntax in a Node.js runtime environment.

3. OBJECTIVE

The objectives of this experiment are:

- To learn array declaration, zero-based indexing, and element mutation in JavaScript.
 - To implement array manipulation methods such as `.push()`, `.shift()`, and evaluate `.length`.
 - To utilize functional programming array methods: `.filter()`, `.map()`, `.find()`, and `.some()`.
 - To understand traditional function declarations, parameter passing, and return values.
 - To implement modern ES6 Arrow Function syntax with implicit and explicit returns.
 - To work with default parameter values, string manipulation (`.trim()`, `.toLowerCase()`), and template literals.
 - To execute scripts in Node.js and analyze terminal output.
-

4. THEORY

JavaScript is a high-level, interpreted or just-in-time compiled language featuring first-class functions. In modern full-stack development, Node.js serves as the server-side runtime executing JavaScript outside the browser using Google Chrome's V8 engine.

Theory of Arrays and Functions:

- **Array Fundamentals:** Arrays are ordered, mutable collections of values. Elements are indexed starting at position 0. JavaScript arrays are dynamically sized and support heterogeneous types.
- **Mutating vs Non-Mutating Methods:** Mutating methods (`.push()`, `.shift()`) alter the original array directly. Non-mutating functional methods (`.map()`, `.filter()`) return new transformed collections.
- **Array Iteration & Searching:** `.find()` returns the first item satisfying a callback predicate, while `.some()` checks whether at least one element fulfills the condition.
- **Functions & Arrow Syntax:** Traditional functions feature dynamic `this` binding and hoisting. ES6 arrow functions (`() => {}`) provide concise syntax and lexical `this` scoping.
- **Parameter Defaults & Template Literals:** Default parameters prevent undefined errors when arguments are omitted. Template literals (backticks) permit multi-line strings and interpolation (`${}`).

5. ALGORITHM / PROCEDURE

1. Create project directories: Record/Experiment - 3(week -3)/Src and Documentation.
 2. Create arrays.js and functions.js in the Src directory.
 3. In arrays.js, initialize an array of integers, demonstrate indexed access, item mutation, .push(), and .shift().
 4. Perform functional array transformations using .filter() to select numbers > 20 and .map() to double each element.
 5. Declare a complex array of product objects representing a shopping cart with id, name, price, and inStock status.
 6. Execute .find() to locate a product by ID, .filter() for in-stock inventory, .some() for price validation, and .map() to extract product names.
 7. In functions.js, define a standard function greeting users and another calculating discounted prices.
 8. Write an arrow function to compute multiplication and another to convert Fahrenheit temperatures to Celsius.
 9. Implement a user handle formatting arrow function demonstrating default parameters and string methods.
 10. Execute node arrays.js and node functions.js in the terminal and verify correct console output.
-

6. PROGRAM

arrays.js (Lines 1-52)

```
// 1. Creating an array of numbers
const numbers = [10, 25, 40, 5, 80, 15];

// 2. Accessing elements by index (starts at 0)
console.log("First number:", numbers[0]); // Output: 10
console.log("Third number:", numbers[2]); // Output: 40

// 3. Modifying an item
numbers[1] = 30; // Changes 25 to 30

// 4. Adding and removing items
numbers.push(100); // Adds 100 to the end
numbers.shift();   // Removes the first item (10)

// 5. Checking the array length
console.log("Total items:", numbers.length);

// 6. Common Array Operations
// Filter numbers greater than 20
const bigNumbers = numbers.filter(num => num > 20);
console.log("Numbers > 20:", bigNumbers);

// Multiply every number by 2
const doubled = numbers.map(num => num * 2);
console.log("Doubled numbers:", doubled);

// Sample array of products in a shopping cart
const cart = [
  { id: 101, name: "Wireless Mouse", price: 25, inStock: true },
  { id: 102, name: "Mechanical Keyboard", price: 85, inStock: false },
  { id: 103, name: "USB-C Cable", price: 12, inStock: true },
  { id: 104, name: "Gaming Monitor", price: 220, inStock: true }
];

// 1. Array.prototype.find() - Find a single item by ID
const item = cart.find(product => product.id === 103);
console.log("Selected Item:", item.name); // Output: USB-C Cable

// 2. Array.prototype.filter() - Get only items that are in stock
const availableItems = cart.filter(product => product.inStock);
console.log("Available Items Count:", availableItems.length); // Output: 3

// 3. Array.prototype.some() - Check if any item exceeds $200
const hasExpensiveItem = cart.some(product => product.price > 200);
console.log("Has item > $200?:", hasExpensiveItem); // Output: true

// 4. Array.prototype.map() - Get an array of just the product names
const itemNames = cart.map(product => product.name);
console.log("Product Names:", itemNames);
```

functions.js (Lines 1-49)

```
// 1. Basic Function declaration
function greetUser(userName) {
  return `Hello, ${userName}! Welcome back.`;
}

// Calling the function
const message = greetUser("Alex");
console.log(message); // Output: Hello, Alex! Welcome back.

// 2. Function with multiple parameters & math calculation
function calculateDiscount(price, discountPercent) {
  const savings = (price * discountPercent) / 100;
  const finalPrice = price - savings;
  return finalPrice;
}

// Calling the calculation function
const laptopPrice = calculateDiscount(1000, 15);
console.log("Final Price: $" + laptopPrice); // Output: Final Price: $850

// 3. Arrow Function syntax (Modern ES6 alternative)
const multiply = (a, b) => a * b;

console.log("Product:", multiply(6, 7)); // Output: Product: 42

// 1. Function to convert Fahrenheit to Celsius
function fahrenheitToCelsius(fahrenheit) {
  const celsius = (fahrenheit - 32) * (5 / 9);
  return celsius.toFixed(1); // Round to 1 decimal place
}

// Calling the conversion function
console.log("75°F in Celsius is:", fahrenheitToCelsius(75), "°C"); // Output: 23.9 °C
console.log("32°F in Celsius is:", fahrenheitToCelsius(32), "°C"); // Output: 0.0 °C

// 2. Arrow function to format user display names with optional default parameters
const formatUserHandle = (username, role = "Member") => {
  const cleanUsername = username.trim().toLowerCase();
  return `@${cleanUsername} [${role.toUpperCase()}]`;
};

// Calling the formatting function
console.log(formatUserHandle(" JohnDoe ", "Admin")); // Output: @johndoe [ADMIN]
console.log(formatUserHandle("Sarah88")); // Output: @sarah88 [MEMBER]
```

7. CODE EXPLANATION

Construct / Method	Explanation
const	Declares a block-scoped variable whose reference cannot be reassigned.
Array Literal []	Initializes an ordered collection of values enclosed in brackets.
Array Indexing [i]	Accesses or modifies the element situated at zero-based index i.
.push(val)	Appends an element to the end of the array and updates its length.
.shift()	Removes the first element from an array and shifts all other elements down.
.length	Property returning the total count of elements in the array.
.filter(callback)	Creates a new array with all elements passing the predicate test.
.map(callback)	Creates a new array populated with the results of invoking a function on every item.
.find(callback)	Returns the first element satisfying the test function, or undefined.
.some(callback)	Returns a boolean indicating whether at least one element passes the test.
function name(params)	Standard JavaScript function declaration with parameters and return value.
=> (Arrow Function)	Modern ES6 concise function syntax with lexical this binding.
.toFixed(n)	Formats a number using fixed-point notation with n decimal places.
.trim()	Strips whitespace from both leading and trailing ends of a string.
.toLowerCase()	Converts all alphabetic characters in a string to lowercase.
Template Literals	String literals enclosed in backticks allowing embedded expressions `\${}`.

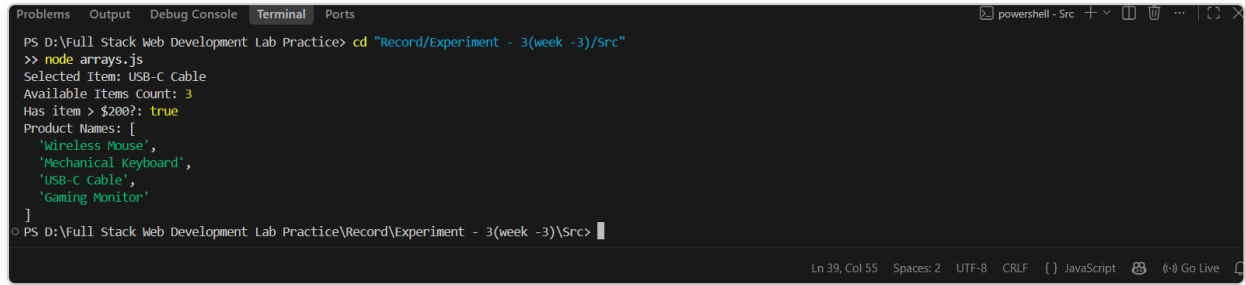
8. TEST CASES AND EXECUTION DATA

The following test suite was executed in Node.js runtime to verify array manipulations and functional operations:

TC ID	Test Description	Input / Action	Expected Result	Status
TC-01	Array Indexing & Mutation	numbers[1] = 30	Element at index 1 updates from 25 to 30	Pass
TC-02	Array Push & Shift	numbers.push(100); numbers.shift()	100 added to end, 10 removed from front; length is 6	Pass
TC-03	Array Filter & Map	filter(num > 20) & map(num * 2)	Returns filtered array [30, 40, 80, 100] and doubled values	Pass
TC-04	Object Array Find & Some	cart.find(id === 103); cart.some(price > 200)	Finds 'USB-C Cable' and validates expensive items exist (true)	Pass
TC-05	Discount Calculation	calculateDiscount(1000, 15)	Correctly computes savings (\$150) and returns final price \$850	Pass
TC-06	Temperature & Handle Formatting	fahrenheitToCelsius(75); formatUserHandle('JohnDoe ', 'Admin')	Computes 23.9 °C and returns formatted string '@johndoe [ADMIN]'	Pass

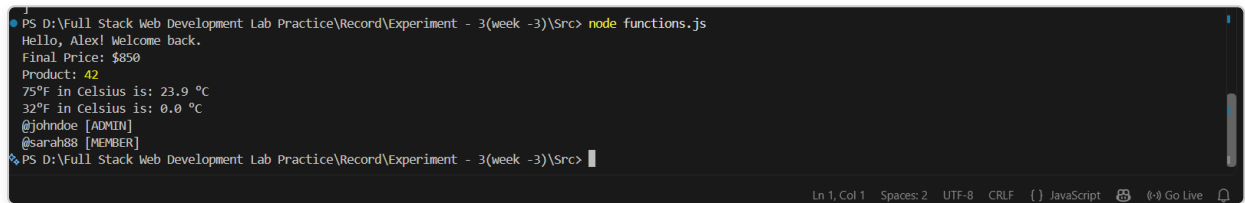
9. OUTPUT

Output: Node.js Terminal Execution Output



```
PS D:\Full Stack Web Development Lab Practice> cd "Record/Experiment - 3(week -3)/Src"
>> node arrays.js
Selected Item: USB-C Cable
Available Items Count: 3
Has item > $200?: true
Product Names: [
  'Wireless Mouse',
  'Mechanical Keyboard',
  'USB-C Cable',
  'Gaming Monitor'
]
```

Figure 1: Node.js terminal output for arrays.js execution.



```
PS D:\Full Stack Web Development Lab Practice\Record\Experiment - 3(week -3)\Src> node functions.js
Hello, Alex! Welcome back.
Final Price: $850
Product: 42
75°F in Celsius is: 23.9 °C
32°F in Celsius is: 0.0 °C
@johndoe [ADMIN]
@sarah88 [MEMBER]
```

Figure 2: Node.js terminal output for functions.js execution.

10. RESULT

Thus, JavaScript arrays, higher-order methods, traditional functions, and ES6 arrow functions were successfully implemented, executed, and verified under Node.js runtime environment.

Submission Package Structure:

```
Experiment - 3(week -3)/
├── Documentation/
│   ├── Experiment_3.docx
│   └── Experiment_3.pdf
├── Screenshot/
│   ├── Screenshot 2026-08-16 224001.png
│   └── Screenshot 2026-08-16 224037.png
└── Src/
    ├── arrays.js
    └── functions.js
```